SERVICIO NACIONAL DE APRENDIZAJE - SENA

Diseño y desarrollo de servicios web - proyecto

CRUD de citas expuesto como API REST, sobre el mismo código del módulo web de Barbería Harry Estilos

Evidencia de aprendizaje GA7-220501096-AA5-EV03

Aprendices:
Hernán Salazar Ospina, Diego Caicedo Saldarriaga, José Arteta Angulo

Instructora: Verónica M. Muñoz
Programa de formación: Análisis y Desarrollo de Software
Proyecto formativo: Barbería Harry Estilos - Módulo de agendamiento
Ficha: 3336210

Colombia, 2026

Introducción

La evidencia GA7-220501096-AA5-EV01 expuso como API REST el módulo de inicio de sesión y registro, reutilizando literalmente las clases del proyecto principal. Esta evidencia hace lo mismo con el módulo de agendamiento: toma el CRUD de citas que ya funciona como flujo web (formulario, servlets, base de datos) y lo expone también como servicio REST, sin tocar el código del sitio web ni reescribir su lógica.

El resultado son dos puertas de entrada distintas al mismo sistema: un navegador que llena un formulario en el puerto 8080, y un cliente HTTP (Postman, una app externa) que llama al puerto 8081. Ambas terminan ejecutando las mismas clases DAO sobre la misma base de datos.

Objetivos

Diseñar y codificar los servicios REST necesarios para el CRUD de citas y las listas de apoyo (barberos, servicios) del proyecto formativo.

Documentar cada servicio: ruta, método HTTP, parámetros de entrada y respuestas.

Mantener el proyecto versionado en el repositorio de control de versiones del equipo.

Análisis de la evidencia

Antes de escribir código se revisó qué ya existe en el repositorio: el proyecto principal (barberia-harry-estilos) tiene, en su paquete com.harryestilos.dao, las clases CitaDAO, BarberoDAO, ServicioDAO y ClienteDAO, con los métodos de inserción, consulta, actualización y eliminación ya probados por el flujo web (AgendamientoServlet, CitasServlet,

EliminarCitaServlet). No había que diseñar el acceso a datos desde cero: había que decidir cómo exponerlo por HTTP.

Aquí conviene anotar algo que cambia el alcance de esta evidencia: el CitaDAO del proyecto no valida cruces de horario entre citas de un mismo barbero. La tabla citas no tiene ninguna restricción que lo impida, y ni el DAO ni los servlets del módulo web comprueban si un barbero ya tiene una cita en ese horario antes de guardar una nueva. Por eso esta API REST respeta el mismo comportamiento: no se agregó una validación que el canal web no tiene, para que los dos canales se comporten igual. Queda anotado como mejora pendiente del proyecto en general.

Diseño de la solución

Decisiones de arquitectura

Tabla 1
Decisiones de arquitectura y su justificación

Decisión	Alternativa considerada	Justificación
Los nuevos endpoints viven en el mismo servicio-auth-api	Crear un módulo Spring Boot aparte solo para citas	Ya existe un servicio corriendo en el puerto 8081 con la infraestructura lista (pom.xml, ConexionDB, convención de paquetes). Separar citas en otro módulo solo duplicaría esa infraestructura.
Los datos llegan como x-www-form-urlencoded	Cuerpo JSON	Mantiene la misma convención que /api/usuario/login y /api/usuario/registro (EV01), para que toda la colección de Postman del proyecto use el mismo formato.
No se agrega validación de cruce de horarios	Implementarla solo en la API, distinta del canal web	Esa regla no existe en el código fuente del proyecto. Agregarla únicamente en la API crearía un comportamiento distinto entre los dos canales sobre la misma base de datos, lo cual es peor que no tenerla en ninguno.

Reutilización de componentes existentes

Siguiendo el mismo patrón de la EV01 (copia literal de UsuarioDAO, Usuario, PasswordUtil y RegistroValidator), se copiaron al módulo servicio-auth-api las siguientes clases del proyecto principal:

Tabla 2
Clases reutilizadas del proyecto principal

Clase	Paquete	Origen
Cita, Barbero, Servicio, Cliente	model	Proyecto principal (com.harryestilos.model)
CitaDAO, BarberoDAO, ServicioDAO, ClienteDAO	dao	Proyecto principal (com.harryestilos.dao)

Especificación y documentación de los servicios

Tabla 3
Catálogos de solo lectura

Endpoint	Método	Parámetros	Respuestas
/api/barberos	GET	-	200 OK - lista de barberos (id, nombre)
/api/servicios	GET	-	200 OK - lista de servicios (id, nombre, clave, precio, duracionMinutos)
/api/servicios/{id}	GET	-	200 OK / 404 Not Found

Tabla 4
CRUD de citas

Endpoint	Método	Parámetros	Respuestas
/api/citas	POST	cliente, barbero_id, servicio_id, fecha, hora	201 Created / 400 Bad Request
/api/citas	GET	-	200 OK - lista de todas las citas
/api/citas/{id}	GET	-	200 OK / 404 Not Found
/api/citas/{id}	PUT	cliente, barbero_id, servicio_id, fecha, hora	200 OK / 400 / 404 Not Found
/api/citas/{id}	DELETE	-	200 OK / 404 Not Found

El código de creación (POST /api/citas) valida que los campos cliente, fecha y hora no estén vacíos y que el servicio_id exista antes de insertar; el cliente se resuelve con ClienteDAO.obtenerOCrear, que reutiliza el cliente si ya existe por nombre o lo crea si es la

primera vez. Ni la creación ni la edición comprueban cruces de horario, por lo coherente con el análisis de la sección anterior.

Base de datos

Se usa la misma base de datos MySQL barberia_harry_estilos del proyecto principal, sin cambios de esquema: las tablas citas, barberos, servicios y clientes ya existían antes de esta evidencia (script init.sql del repositorio). No se agregaron columnas ni tablas nuevas.

Herramientas de versionamiento

El código se desarrolló y se versionó con Git sobre el repositorio compartido del equipo en GitHub (https://github.com/dfcaicedosaldarriaga-cmd/barberia-harry-estilos), dentro de la carpeta servicio-auth-api ya existente desde la EV01.

Conclusiones

Exponer el CRUD de citas como servicio REST no exigió diseñar de nuevo el acceso a datos: la decisión de diseño de la EV01 (reutilizar clases literalmente en lugar de reescribirlas) se sostuvo también aquí. Lo más relevante de esta evidencia terminó siendo de honestidad en la documentación más que técnico: identificar que el proyecto no valida cruces de horario, y decidir no simularlo solo en la capa nueva.

Referencias

Servicio Nacional de Aprendizaje [SENA]. (2024). Guía de aprendizaje 7 - GA7-220501096: Codificar el software de acuerdo con el diseño establecido.

https://zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/Tecnologia/228118_1/Contenido/DocArtic/GUI7/Guia_aprendizaje_7.pdf

VMware (Spring). (s.f.). Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/